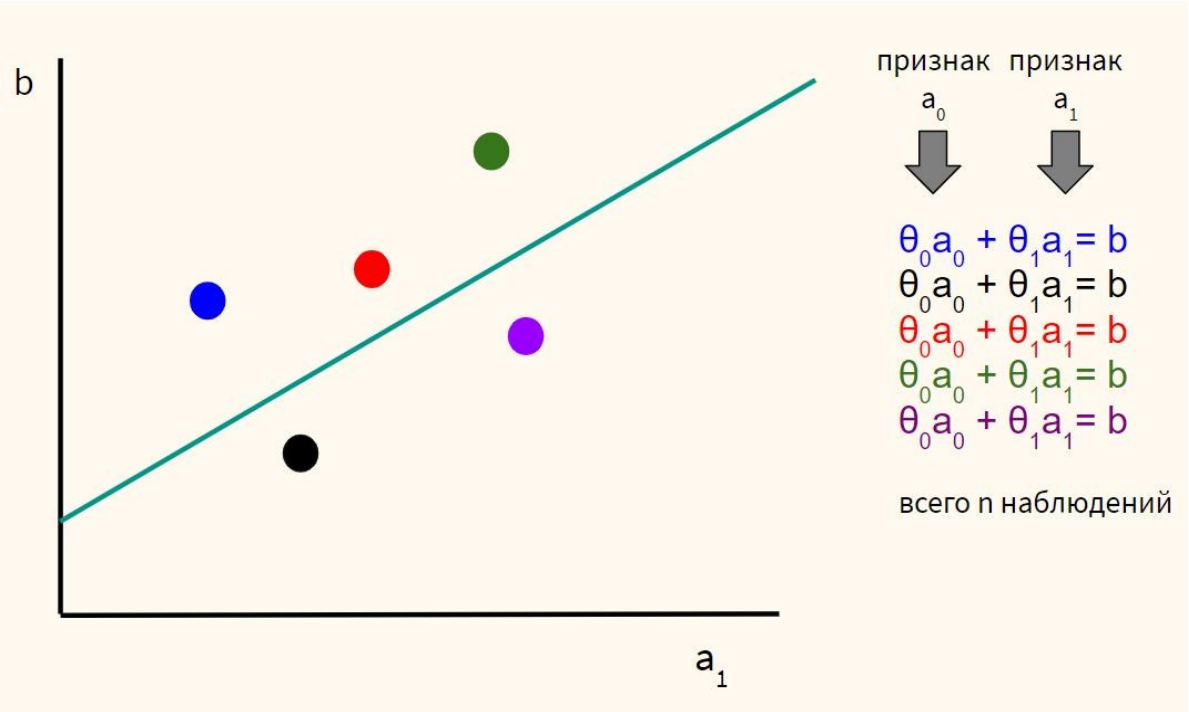
Линейная регрессия. Часть 3

>

Продолжим изучать линейную регрессию.

Линейная регрессия как проекция

Представим данные как систему линейных уравнений.



На рисунке выше представлена обычная задача простой линейной регрессии с немного измененной нотацией. У нас есть пять наблюдений, каждое наблюдение описано признаками a_0 и a_1, при

этом a_0 заполнен единицами. Подставив некоторые веса θ_0 и θ_1, мы получим для каждой точки значение целевой переменной b (ранее мы ее обозначали через y).

Единственного решения для такой системы не существует, поскольку мы не сможем подобрать такие θ_0 и θ_1, которые бы удовлетворяли всем a_0 и a_1 в каждой строке (для каждого наблюдения).

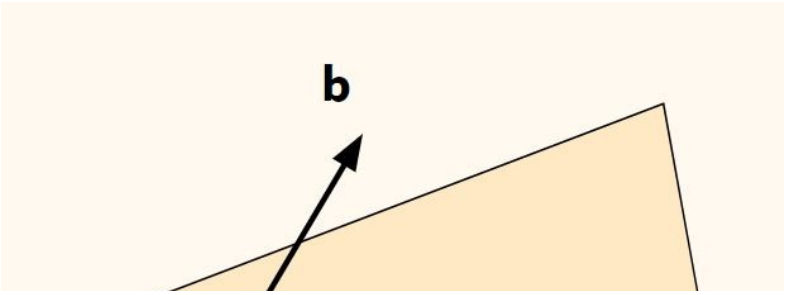
Такая система уравнений называется **переопределенной** (overdetermined). Она содержит пять уравнений (наблюдений) и при двух неизвестных (коэффициентах). Можно также сказать, что мы наложили слишком много ограничений (ограничения, в данном случае, это уравнения), чтобы найти единственное решение.

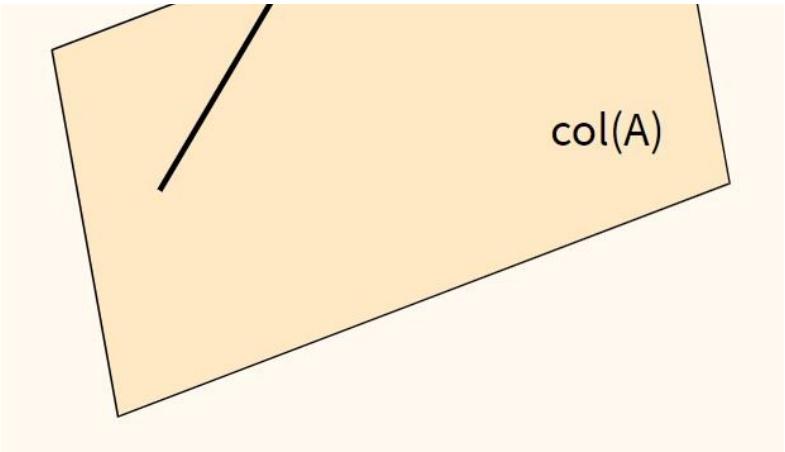
Перепишем эту систему с помощью векторов и матриц. В общем случае у нас конечно может быть больше признаков $\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2, \ldots, \mathbf{a}_d$, однако систему по-прежнему будем считать переопределенной, то есть $n > d$. Также заменим $\boldsymbol{\theta}$ на $\mathbf{x}$.

$$\begin{bmatrix} \vdots & \vdots & \vdots & \vdots \\ \mathbf{a}_1 & \mathbf{a}_2 & \ldots & \mathbf{a}_d \\ \vdots & \vdots & \vdots & \vdots \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$\underset{n \times d}{\mathbf{A}}\, \mathbf{x} = \mathbf{b}$$

При этом в данном случае $\mathbf{A}, \mathbf{x} \in \mathbb{R}^d$, а $\mathbf{b} \in \mathbb{R}^n$. То есть вектор $\mathbf{b}$ находится в пространстве большей размерности, чем $\mathbf{x}$. Например, вектор $\mathbf{x}$ может находиться на плоскости (двумерный вектор), а вектор $\mathbf{b}$ в трехмерном пространстве.





Можно также сказать, что не существует линейной комбинации значений вектора $\mathbf{x}$ (веса модели) и векторов $\mathbf{a}_0, \mathbf{a}_1, \mathbf{a}_2, .., \mathbf{a}_d$, которые преобразовывались бы в вектор $\mathbf{b}$.

$$x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + x_3\mathbf{a}_3 + \ldots + x_d\mathbf{a}_d \neq \mathbf{b}$$

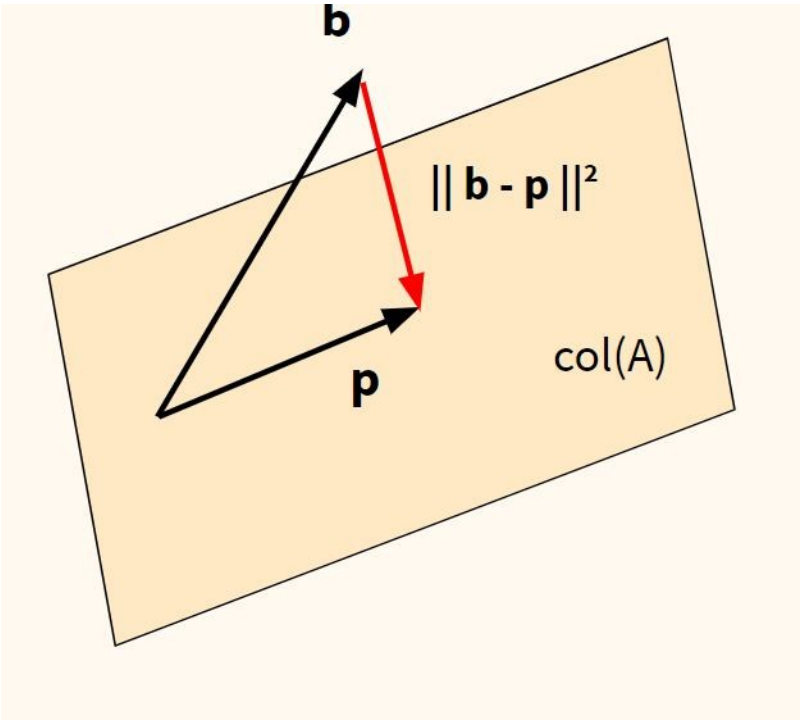
Наконец, справедливо, что $\mathbf{b}$ не находится в пространстве столбцов $\mathbf{A}, \mathbf{b} \notin col(\mathbf{A})$ (то есть не находится на плоскости), а вектор $\mathbf{x}$ как раз лежит на этой плоскости, и это значит, что мы никак не можем перевести с помощью матрицы $\mathbf{A}$ вектор $\mathbf{x}$ из двумерного пространства в трехмерное.

С другой стороны, мы можем попробовать получить наилучшее возможное решение, найдя такой вектор $\mathbf{x}^*$, который будет максимально приближен к вектору $\mathbf{b}$ (т.е. будет иметь *минимальное* расстояние до него). Расстояние же между векторами можно определить как разницу двух векторов.

Для того чтобы положительные и отрицательные значения не взаимоудалялись, возведем значения в квадрат.

$$min||\mathbf{b} - \mathbf{A}\mathbf{x}^*||^2$$

Очевидно, что вектор (назовем его $\mathbf{p}$), получившийся в результате $\mathbf{A}\mathbf{x}^*$ будет в пространстве $\mathbb{R}^d$, что то же самое, что $\mathbf{p} \in col(\mathbf{A})$ (то есть в данном случае на плоскости).



Далее, наименьшее расстояние от вектора $\mathbf{b}$ до $\mathbf{p}$ можно определить как ортогональную проекцию $\mathbf{b}$ на пространство столбцов $\mathbf{A}$.

$$\mathbf{A}\mathbf{x}^* = \mathbf{p} = proj_{col(A)}\mathbf{b}$$

Попробуем найти решение относительно $\mathbf{b}$.

$$\mathbf{A}\mathbf{x}^* = proj_{col(A)}\mathbf{b}$$

Вычтем вектор $\mathbf{b}$ из обеих частей.

$$\mathbf{A}\mathbf{x}^* - \mathbf{b} = proj_{col(A)}\mathbf{b} - \mathbf{b}$$

Заметим, что вектор $proj_{col(A)}\mathbf{b} - \mathbf{b}$ (на рисунке представлен красным вектором) ортогонален к плоскости $col(\mathbf{A})$. Как следствие, $\mathbf{A}\mathbf{x}^* - \mathbf{b}$ ортогонально $col(\mathbf{A})$.

Можно также сказать, что $\mathbf{A}\mathbf{x}^* - \mathbf{b}$ является ортогональным дополнением пространства $col(\mathbf{A})$. Запишем это как $\mathbf{A}\mathbf{x}^* - \mathbf{b} = col(\mathbf{A})^\perp$.

Одновременно ортогональное дополнение пространства столбцов матрицы $\mathbf{A}$ равно ядру $\mathbf{A}^\top$, то есть $col(\mathbf{A})^\perp = null(\mathbf{A}^\top)$. Тогда,

$$\mathbf{A}\mathbf{x}^* - \mathbf{b} \in null(\mathbf{A}^\top)$$

Если умножить матрицу $\mathbf{A}^\top$ на ее ядро $\mathbf{A}\mathbf{x}^* - \mathbf{b}$, то мы получим нулевой вектор.

$$\mathbf{A}^\top(\mathbf{A}\mathbf{x}^* - \mathbf{b}) = \mathbf{0}$$

$$\mathbf{A}^\top\mathbf{A}\mathbf{x}^* - \mathbf{A}^\top\mathbf{b} = \mathbf{0}$$

$$\mathbf{A}^\top\mathbf{A}\mathbf{x}^* = \mathbf{A}^\top\mathbf{b}$$

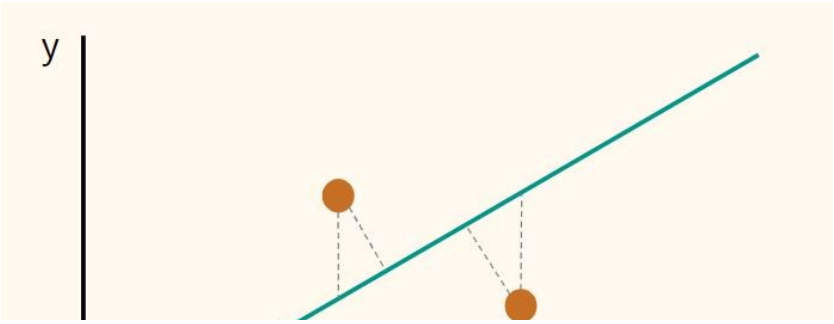
Таким образом можно найти $\mathbf{x}^*$, которое минимизирует квадрат расстояния между вектором $\mathbf{b}$ и вектором проекции $\mathbf{A}\mathbf{x}^*$.

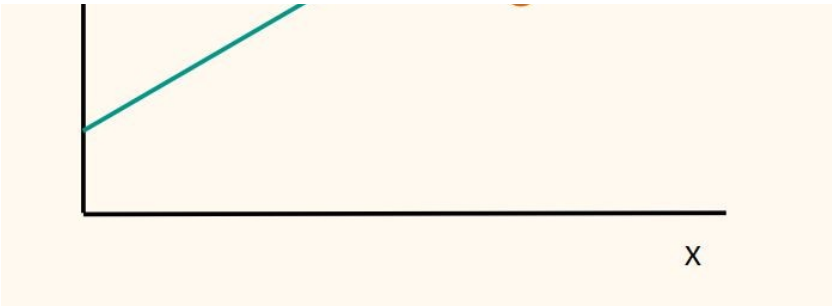
Уверен, вы узнали в этом выражении нормальные уравнения, о которых мы говорили на занятии по линейной регрессии.

$$\mathbf{X}^\top\mathbf{X}\boldsymbol{\theta} = \mathbf{X}^\top\mathbf{y}$$

Нормальными же эти уравнения называются, потому что $\mathbf{A}\mathbf{x}^* - \mathbf{b} \perp col(\mathbf{A})$, а нормалью в геометрии как раз считается обобщенное понятие перпендикуляра к поверхности.

Более того, можно сказать, что минимизация расстояний от точек до прямой вдоль оси y одновременно приводит к минимизации длины перпендикуляров к проекциям точек.





Пример

Возьмем систему уравнений.

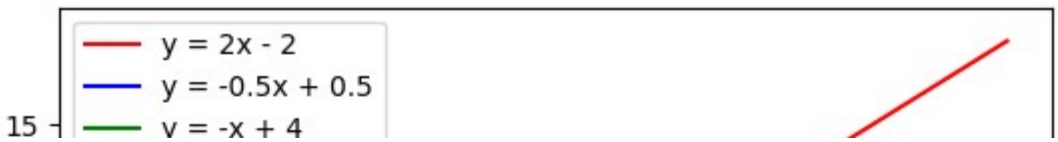
$$\begin{cases} 2x & - & y & = & 2 \\ x & + & 2y & = & 1 \\ x & + & y & = & 4 \end{cases}$$

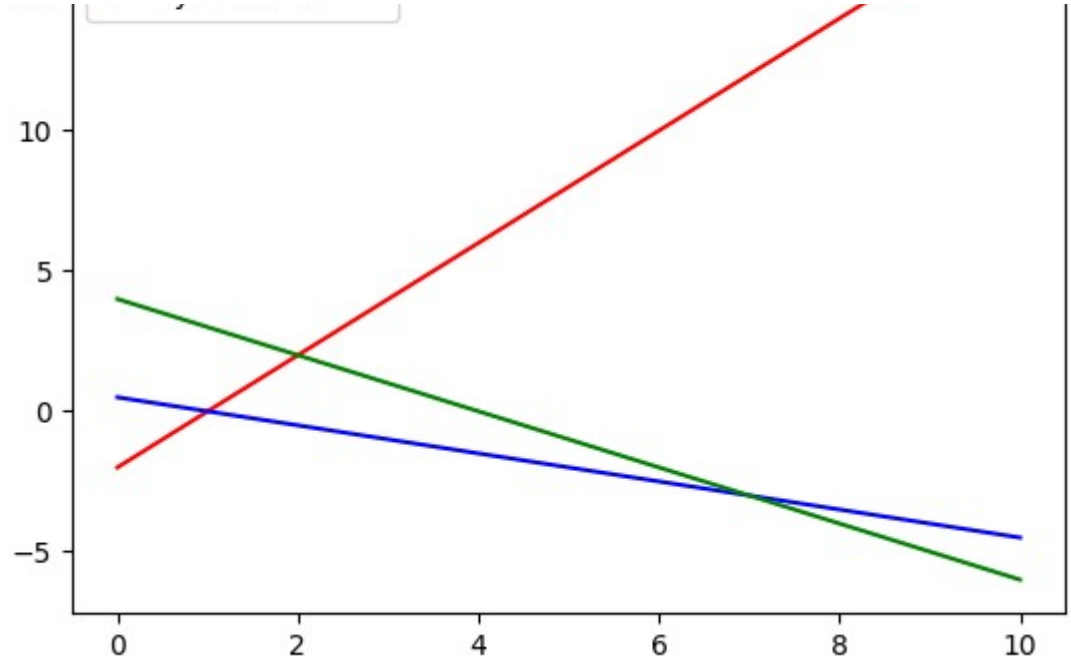
Такая система опять же могла бы описывать данные, где x и y представляют собой некоторые коэффициенты (без сдвига, например, θ_1 и θ_2), а сами уравнения — объекты (наблюдения).

Убедимся, что нам не удастся найти x и y, которые бы удовлетворяли каждому из уравнений, построив соответствующие прямые на графике (для этого решим уравнения относительно y).

Продолжим работу в том же блокноте

```
x = np.linspace(0, 10, 100)

plt.plot(x, 2 * x - 2, 'r', label = 'y = 2x - 2')
plt.plot(x, -0.5 * x + 0.5, 'b', label = 'y = -0.5x + 0.5')
plt.plot(x, -x + 4, 'g', label = 'y = -x + 4')

plt.legend()
plt.show()
```





Как мы видим, нет точки, в которой линии бы пересекались.

Перепишем систему в виде матрицы и вектора.

```
A = np.array([[2, -1],
              [1, 2],
              [1, 1]])

b = np.array([2, 1, 4])
```

Опять же решить систему методом Гаусса у нас не получится. То есть $\mathbf{A}\mathbf{x} = \mathbf{b}$ решения не имеет. Однако мы можем найти приближенное решение через $\mathbf{A}^\top\mathbf{A}\mathbf{x}^* = \mathbf{A}^\top\mathbf{b}$. Найдем $\mathbf{A}^\top\mathbf{A}$ и $\mathbf{A}^\top\mathbf{b}$.

```
A_T_A = np.dot(A.T, A)
A_T_A
```

```
array([[6, 1],
       [1, 6]])
```

```
A_T_b = np.dot(A.T, b)
A_T_b
```

```
array([9, 4])
```

Таким образом, мы получили

$$\begin{bmatrix} 6 & 1 \\ 1 & 6 \end{bmatrix} \mathbf{x}^* = \begin{bmatrix} 9 \\ 4 \end{bmatrix}$$

Решим систему (а такая система уже решается) методом Гаусса. Применим следующие элементарные преобразования:

- $R_1 = \frac{R_1}{6}$;
- $R_2 = R_2 - R_1$;
- $R_2 = \frac{6R_2}{35}$;
- $R_1 = R_1 - \frac{R_2}{6}$.

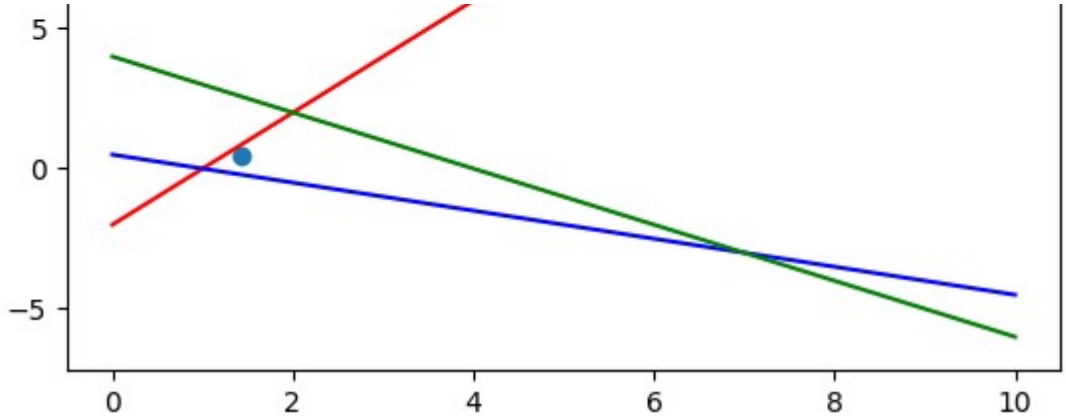
В результате получим следующую матрицу

$$\begin{bmatrix} 1 & 0 & \frac{10}{7} \\ 0 & 1 & \frac{3}{7} \end{bmatrix} \approx \begin{bmatrix} 1 & 0 & 1{,}43 \\ 0 & 1 & 0{,}43 \end{bmatrix}$$

Таким образом, вектор $\begin{bmatrix} 1{,}43 \\ 0{,}43 \end{bmatrix}$ будет решением этой системы уравнений.

Обозначим его на графике.

```
x = np.linspace(0, 10, 100)

plt.plot(x, 2 * x - 2, 'r', label = 'y = 2x - 2')
plt.plot(x, -0.5 * x + 0.5, 'b', label = 'y = -0.5x + 0.5')
plt.plot(x, -x + 4, 'g', label = 'y = -x + 4')

plt.scatter(1.43, 0.43)

plt.legend()
plt.show()
```



Решим эту задачу на Питоне. Вначале с помощью функции **np.linalg.lstsq()**.

```
np.linalg.lstsq(A, b, rcond = None)[0].round(2)
```

```
array([1.43, 0.43])
```

Теперь с помощью класса LinearRegression библиотеки sklearn (напомню, что этот класс также использует метод наименьших квадратов).

```
from sklearn.linear_model import LinearRegression

LinearRegression(fit_intercept = False).fit(A, b).coef_.round(2
```

```
array([1.43, 0.43])
```

Дополнительно проверим, получим ли мы такой же результат с помощью метода градиентного спуска, а заодно убедимся, что метод наименьших квадратов одновременно минимизирует расстояние от точек до прямой вдоль оси y.

```
class gd():
  def __init__(self):
    self.thetas = None
    self.loss_history = []

  def add_ones(self, x):
    return np.c_[np.ones((len(x), 1)), x]

  def objective(self, x, y, thetas, n):
    return np.sum((y - self.h(x, thetas)) ** 2) / (2 * n)

```

```python
  def h(self, x, thetas):
    return np.dot(x, thetas)

  def gradient(self, x, y, thetas, n):
    return np.dot(-x.T, (y - self.h(x, thetas))) / n

  def fit(self, x, y, iter = 20000, learning_rate = 0.05):
    x, y = x.copy(), y.copy()
    self.add_ones(x)

    thetas, n = np.zeros(x.shape[1]), x.shape[0]

    loss_history = []

    for i in range(iter):
      loss_history.append(self.objective(x, y, thetas, n))
      grad = self.gradient(x, y, thetas, n)
      thetas -= learning_rate * grad

    self.thetas = thetas
    self.loss_history = loss_history

  def predict(self, x):
    x = x.copy()
    self.add_ones(x)
    return np.dot(x, self.thetas)
```

```python
model = gd()
# для таких несложных данных должно хватить и ста итераций
model.fit(A, b, iter = 100, learning_rate = 0.05)

model.thetas.round(2)
```

```
array([1.43, 0.43])
```

Обратимость матрицы $\mathbf{A}^\top\mathbf{A}$

Общее решение требует нахождения обратной матрицы $(\mathbf{A}^\top\mathbf{A})^{-1}$.

$$\mathbf{x} = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b}$$

Посмотрим, в каких случаях матрица $\mathbf{A}^\top\mathbf{A}$ обратима.

Такая матрица будет обратима, если столбцы $\mathbf{A}$ линейно независимы (то есть матрица имеет полный ранг).

Приведем простой пример двух матриц $\mathbf{A}_1$ и $\mathbf{A}_2$ размерностью 3×2 с линейно независимыми (полный ранг) и линейно зависимыми столбцами (ранг 1).

$$\mathbf{A}_1 = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 5 \end{bmatrix}, \quad \mathbf{A}_2 = \begin{bmatrix} 1 & 3 \\ 1 & 3 \\ 1 & 3 \end{bmatrix}$$

Найдем $\mathbf{A}_1^\top \mathbf{A}_1$ и $\mathbf{A}_2^\top \mathbf{A}_2$.

$$\mathbf{A}_1^\top \mathbf{A}_1 = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 5 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 5 \end{bmatrix} = \begin{bmatrix} 3 & 8 \\ 8 & 30 \end{bmatrix}$$

$$\mathbf{A}_2^\top \mathbf{A}_2 = \begin{bmatrix} 1 & 1 & 1 \\ 3 & 3 & 3 \end{bmatrix} \begin{bmatrix} 1 & 3 \\ 1 & 3 \\ 1 & 3 \end{bmatrix} = \begin{bmatrix} 3 & 9 \\ 9 & 27 \end{bmatrix}$$

Очевидно, в первом случае матрица обратима, во втором — нет. Приведем и более формальное доказательство.

Предположим, что $\mathbf{A}^\top \mathbf{A}\mathbf{x} = \mathbf{0}$. Тогда, чтобы столбцы $\mathbf{A}^\top \mathbf{A}$ были линейно независимы, должно выполняться условие, при котором ядро содержит только нулевой вектор $null(\mathbf{A}) = \{\mathbf{0}\}$ и $\mathbf{x} = \mathbf{0}$. Умножим обе части на $\mathbf{x}^\top$.

$$\mathbf{x}^\top \mathbf{A}^\top \mathbf{A}\mathbf{x} = \mathbf{x}^\top \mathbf{0}$$

Получим умножение двух векторов (их результатом будет скаляр, а не нулевой вектор).

$$(\mathbf{A}\mathbf{x})^\top (\mathbf{A}\mathbf{x}) = 0$$

$$||\mathbf{Ax}||^2 = 0$$

Значит,

$$\mathbf{Ax} = \mathbf{0}$$

Так как из $\mathbf{A}^\top\mathbf{Ax} = \mathbf{0}$ следует, что $\mathbf{Ax} = \mathbf{0}$, то можно сказать, что если $\mathbf{x} \in null(\mathbf{A}^\top\mathbf{A})$, то $\mathbf{x} \in null(\mathbf{A})$. Отсюда

$$null(\mathbf{A}^\top\mathbf{A}) = null(\mathbf{A})$$

Из этого следует, что

$$dim(null(\mathbf{A}^\top\mathbf{A})) = dim(null(\mathbf{A})) \implies rank(\mathbf{A}^\top\mathbf{A}) = rank(\mathbf{A})$$

В целом, как известно, матрица обратима, если она представляет собой квадратную матрицу, и ее столбцы независимы (она имеет полный ранг). Мы знаем, что $\mathbf{A}^\top\mathbf{A}$ квадратная и имеет тот же ранг, что и $\mathbf{A}$. Соответственно, если $\mathbf{A}$ имеет полный ранг, то $\mathbf{A}^\top\mathbf{A}$ обратима.

Добавлю, что если матрицу $\mathbf{A}^\top\mathbf{A}$ возможно привести к упрощенному ступенчатому виду (единичной матрице), т.е. $rref(\mathbf{A}^\top\mathbf{A}) = \mathbf{I}$, то она обратима.

Про матрицу проекции

Выше мы сказали, что $\mathbf{p} = \mathbf{Ax}^*$. Одновременно, решив нормальные уравнения получим $\mathbf{x}^* = (\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top\mathbf{b}$ Как следствие,

$$\mathbf{p} = \mathbf{Ax}^* = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top$$

Свойства матрицы проекции

Так мы получили матрицу проекции (projection matrix). Обозначим ее как $\mathbf{P}$. Посмотрим на несколько интересных свойств. Во-первых, такая

матрица **симметрична**

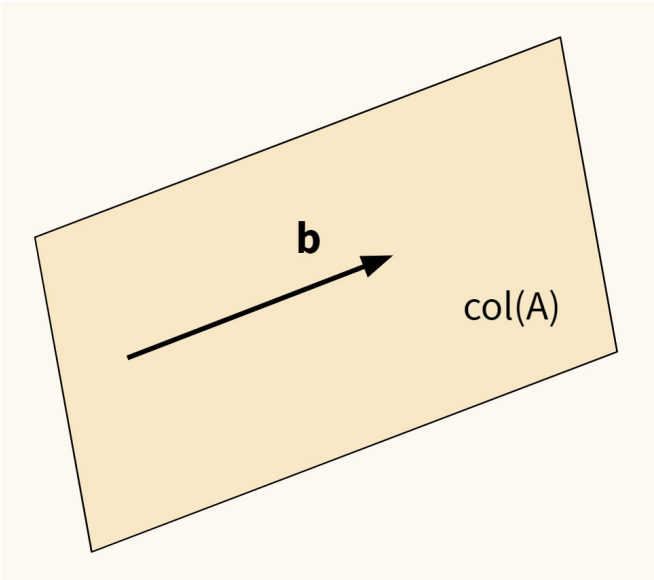
$$\mathbf{P}^\top = \mathbf{P}$$

Во-вторых, **квадрат матрицы** проекции равен самой матрице проекции $\mathbf{P}^2 = \mathbf{P}$.

$$\mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\,\mathbf{A}^\top\mathbf{A}\;\cancel{(\mathbf{A}^\top\mathbf{A})^{-1}}\,\mathbf{A}^\top = \mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top$$

Это же справедливо для любого количества степеней.

$\mathbf{b} \in col(\mathbf{A})$



Что интересно, если $\mathbf{Ax} = \mathbf{b}$ имеет решение, то есть $\mathbf{b} \in col(\mathbf{A})$, то $\mathbf{A}$ — будет квадратной обратимой матрицей, а значит, по свойству $(\mathbf{AB})^{-1} = \mathbf{A}^{-1}\mathbf{B}^{-1}$,

$$\mathbf{A}\mathbf{A}^{-1}(\mathbf{A}^\top)^{-1}\mathbf{A}^\top = \mathbf{I}$$

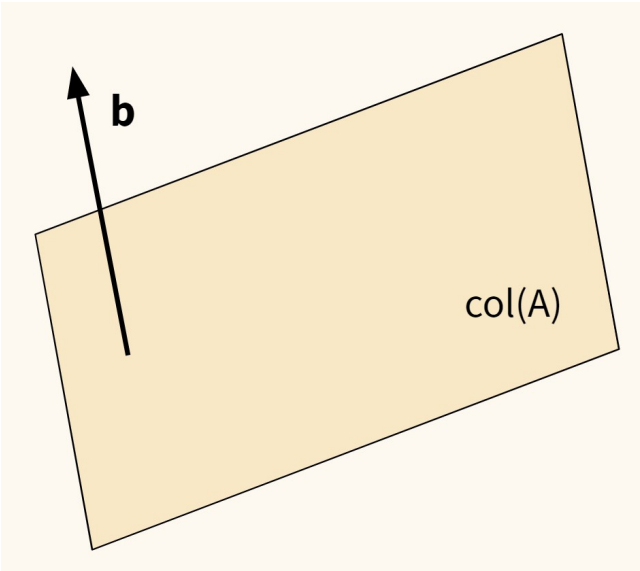
Другими словами, если $\mathbf{Ax} = \mathbf{b}$ проецирует $\mathbf{b}$ на $\mathbf{A}$ и $\mathbf{b} \in col(\mathbf{A})$, то проекцией будет сам вектор $\mathbf{b}$, а скалярной проекцией единица (единичная матрица).

$$\mathbf{Pb} = \mathbf{Ib} = \mathbf{b}$$

Можно также сказать, что $\mathbf{Ax} = \mathbf{b}$,

$$\mathbf{p} = \mathbf{Pb} = (\mathbf{A}(\mathbf{A}^\top\mathbf{A})^{-1}\mathbf{A}^\top)\mathbf{Ax} = \mathbf{A}\,\cancel{(\mathbf{A}^\top\mathbf{A})^{-1}}\,\cancel{(\mathbf{A}^\top\mathbf{A})}\,\mathbf{x} = \mathbf{Ax}$$

$\mathbf{b} \perp col(\mathbf{A})$



Если вектор $\mathbf{b}$ ортогонален пространству столбцов $col(\mathbf{A})$, то он находится в ядре $\mathbf{A}^\top$. Логично, что проекция $\mathbf{b}$ на $col(\mathbf{A})$ представляет собой нулевой вектор.

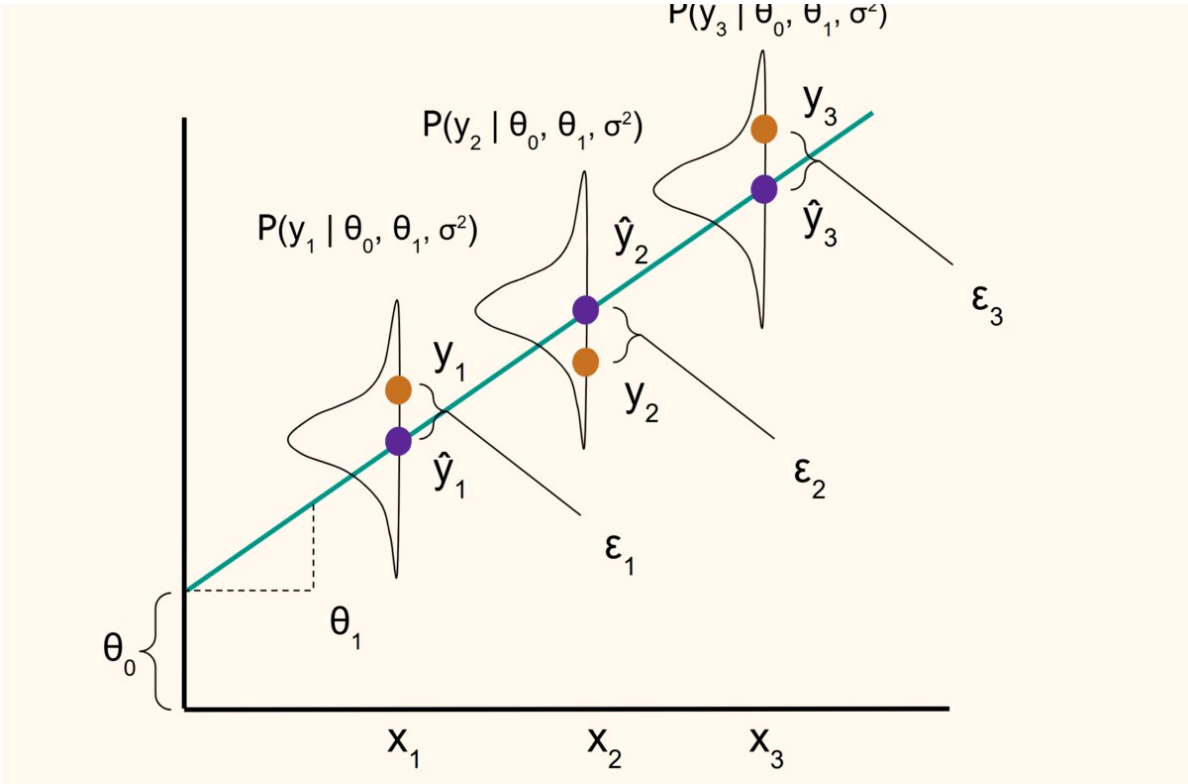
$$\mathbf{Pb} = \mathbf{0}$$

Метод максимального правдоподобия

Рассмотрим модель простой линейной регрессии с независимыми одинаково распределенными наблюдениями.

$$y_i = \theta_0 + \theta_1 x_i + \varepsilon_i,\ \varepsilon_i \sim \mathcal{N}(0, \sigma^2),\ i = 1, \ldots, n$$

Проиллюстрируем эту модель для трех наблюдений x_1, x_2, x_3.



Применим **метод максимального правдоподобия** (maximum likelihood estimation) для оценки параметров θ_0, θ_1 и σ^2.

$$
\begin{aligned}
P(y \mid \theta_0, \theta_1, \sigma^2) &= \prod_{i=1}^{n} P(y_i \mid \theta_0, \theta_1, \sigma^2) \\
&= \prod_{i=1}^{n} \mathcal{N}(y_i; \theta_0 + \theta_1 x_i, \sigma^2) \\
&= \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi\sigma}} \cdot \exp\left[-\frac{(y_i - \theta_0 - \theta_1 x_i)^2}{2\sigma^2}\right] \\
&= \frac{1}{\sqrt{(2\pi\sigma^2)^n}} \cdot \exp\left[-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta_0 - \theta_1 x_i)^2\right]
\end{aligned}
$$

Возьмем логарифм от функции правдоподобия.

$$
\begin{aligned}
\ell(\theta_0, \theta_1, \sigma^2) &= \log P(y \mid \theta_0, \theta_1, \sigma^2) \\
&= -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \theta_0 — \theta_1 x_i)^2
\end{aligned}
$$

Найдем производную ℓ относительно θ_0.

$$\frac{\mathrm{d}\ell(\theta_0, \theta_1, \sigma^2)}{\mathrm{d}\theta_0} = \frac{1}{\sigma^2} \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)$$

Приравняем к нулю и выразим θ_0.

$$\frac{\mathrm{d}\ell(\hat{\theta}_0, \hat{\theta}_1, \hat{\sigma}^2)}{\mathrm{d}\theta_0} = 0$$

$$0 = \frac{1}{\hat{\sigma}^2} \sum_{i=1}^{n} (y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i)$$

$$0 = \sum_{i=1}^{n} y_i - n\hat{\theta}_0 - \hat{\theta}_1 \sum_{i=1}^{n} x_i \; \hat{\theta}_0 = \frac{1}{n} \sum_{i=1}^{n} y_i - \hat{\theta}_1 \frac{1}{n} \sum_{i=1}^{n} x_i$$

$$\hat{\theta}_0 = \bar{y} - \hat{\theta}_1 \bar{x}$$

Найдем производную ℓ со значением $\hat{\theta}_0$ относительно θ_1.

$$\frac{\mathrm{d}\ell(\hat{\theta}_0, \theta_1, \sigma^2)}{\mathrm{d}\theta_1} = \frac{1}{\sigma^2} \sum_{i=1}^{n} (x_i y_i - \hat{\theta}_0 x_i - \theta_1 x_i^2)$$

Приравняем к нулю и выразим θ_1.

$$\frac{\mathrm{d}\ell(\hat{\theta}_0, \hat{\theta}_1, \hat{\sigma}^2)}{\mathrm{d}\theta_1} = 0$$

$$0 = \frac{1}{\hat{\sigma}^2} \sum_{i=1}^{n} (x_i y_i - \hat{\theta}_0 x_i - \hat{\theta}_1 x_i^2)$$

$$0 = \sum_{i=1}^{n} x_i y_i - \hat{\theta}_0 \sum_{i=1}^{n} x_i - \hat{\theta}_1 \sum_{i=1}^{n} x_i^2)$$

$$0 = \sum_{i=1}^{n} x_i y_i - (\bar{y} - \hat{\theta}_1 \bar{x}) \sum_{i=1}^{n} x_i - \hat{\theta}_1 \sum_{i=1}^{n} x_i^2$$

$$0 = \sum_{i=1}^{n} x_i y_i - \bar{y} \sum_{i=1}^{n} x_i + \hat{\theta}_1 \bar{x} \sum_{i=1}^{n} x_i - \hat{\theta}_1 \sum_{i=1}^{n} x_i^2$$

$$0 = \sum_{i=1}^{n} x_i y_i - n\bar{x}\bar{y} + \hat{\theta}_1 n \bar{x}^2 - \hat{\theta}_1 \sum_{i=1}^{n} x_i^2$$

$$\hat{\theta}_1 = \frac{\sum_{i=1}^{n} x_i y_i - \sum_{i=1}^{n} \bar{x}\bar{y}}{\sum_{i=1}^{n} x_i^2 - \sum_{i=1}^{n} \bar{x}^2}$$

$$\hat{\theta}_1 = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2}$$

Как мы видим, метод максимального правдоподобия дал тот же результат, что и аналитический метод.

Осталось получить MLE-оценку σ^2. Найдем производную ℓ со значением $(\hat{\theta}_0, \hat{\theta}_1)$ относительно θ_1.

$$\frac{\mathrm{d}\ell(\hat{\theta}_0, \hat{\theta}_1, \sigma^2)}{\mathrm{d}\sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum_{i=1}^{n} (y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i)^2$$

Приравняем производную к нулю и выразим σ^2.

$$
\begin{aligned}
\frac{\mathrm{d}\ell(\hat{\theta}_0, \hat{\theta}_1, \hat{\sigma}^2)}{\mathrm{d}\sigma^2} &= 0 \\
0 &= -\frac{n}{2\hat{\sigma}^2} + \frac{1}{2(\hat{\sigma}^2)^2} \sum_{i=1}^{n} (y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i)^2 \\
\frac{n}{2\hat{\sigma}^2} &= \frac{1}{2(\hat{\sigma}^2)^2} \sum_{i=1}^{n} (y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i)^2 \\
\hat{\sigma}^2 &= \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{\theta}_0 - \hat{\theta}_1 x_i)^2
\end{aligned}
$$

Теперь рассмотрим модель множественной линейной регрессии с независимыми одинаково распределенными наблюдениями

$$
\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \varepsilon, \varepsilon \sim \mathcal{N}(0, \sigma^2 \mathbf{I}),
$$

где остатки следуют многомерному нормальному распределению с нулевым средним и дисперсией, равной $\sigma^2 \mathbf{I}$.

$$
\begin{aligned}
P(\mathbf{y} \mid \boldsymbol{\theta}, \sigma^2) &= \mathcal{N}(\mathbf{y}; \mathbf{X}\boldsymbol{\theta}, \sigma^2 \mathbf{I}) \\
&= \sqrt{\frac{1}{(2\pi)^n |\sigma^2 \mathbf{I}|}} \cdot \exp\left[-\frac{1}{2}(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top (\sigma^2 \mathbf{I})^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta})\right]
\end{aligned}
$$

Используя свойство умножения определителя матрицы на скаляр $|\sigma^2 \mathbf{I}| = (\sigma^2)^n |\mathbf{I}|$, зададим логарифмическую функцию правдоподобия.

$$
\begin{aligned}
\ell(\boldsymbol{\theta}, \sigma^2) &= \log P(\mathbf{y} \mid \boldsymbol{\theta}, \sigma^2) \\
&= -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2}\log(|\mathbf{I}|) \\
&\quad - \frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{I}^{-1} (\mathbf{y} - \mathbf{X}\boldsymbol{\theta}) \\
&= -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2}\log(|\mathbf{I}|) \\
&\quad - \frac{1}{2\sigma^2}\left(\mathbf{y}^\top \mathbf{I}^{-1} \mathbf{y} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X}\boldsymbol{\theta}\right)
\end{aligned}
$$

Найдем производную ℓ относительно $\boldsymbol{\theta}$.

$$
\begin{aligned}
\frac{\partial \ell(\boldsymbol{\theta}, \sigma^2)}{\partial \boldsymbol{\theta}} &= \frac{\partial}{\partial \boldsymbol{\theta}} \left(-\frac{1}{2\sigma^2} \left(\mathbf{y}^\top \mathbf{I}^{-1} \mathbf{y} - 2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \boldsymbol{\theta} \right) \right) \\
&= \frac{1}{2\sigma^2} \frac{\partial}{\partial \boldsymbol{\theta}} \left(2\boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} - \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \boldsymbol{\theta} \right) \\
&= \frac{1}{2\sigma^2} \left(2\mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} - 2\mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \boldsymbol{\theta} \right) \\
&= \frac{1}{\sigma^2} \left(\mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} - \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \boldsymbol{\theta} \right)
\end{aligned}
$$

Приравняем производную к нулю и выразим $\boldsymbol{\theta}$.

$$
\begin{aligned}
\frac{\partial \ell(\hat{\boldsymbol{\theta}}, \sigma^2)}{\partial \boldsymbol{\theta}} &= 0 \\
0 &= \frac{1}{\sigma^2} \left(\mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} - \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \hat{\boldsymbol{\theta}} \right) \\
0 &= \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} - \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \hat{\boldsymbol{\theta}} \\
\mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \hat{\boldsymbol{\theta}} &= \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} \\
\hat{\boldsymbol{\theta}} &= \left(\mathbf{X}^\top \mathbf{I}^{-1} \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{I}^{-1} \mathbf{y} \\
\hat{\boldsymbol{\theta}} &= \left(\mathbf{X}^\top \mathbf{X} \right)^{-1} \mathbf{X}^\top \mathbf{y}
\end{aligned}
$$

Найдем производную функции ℓ со значением $\hat{\boldsymbol{\theta}}$ относительно σ^2.

$$
\begin{aligned}
\frac{\partial \ell(\hat{\boldsymbol{\theta}}, \sigma^2)}{\partial \sigma^2} &= \frac{\partial}{\partial \sigma^2} \left(-\frac{n}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}) \right) \\
&= -\frac{n}{2} \frac{1}{\sigma^2} + \frac{1}{2(\sigma^2)^2} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}) \\
&= -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})
\end{aligned}
$$

Приравняем производную к нулю и выразим σ^2.

$$
\begin{aligned}
\frac{\partial \ell(\hat{\boldsymbol{\theta}}, \hat{\sigma}^2)}{\partial \sigma^2} &= 0 \\
0 &= -\frac{n}{2\hat{\sigma}^2} + \frac{1}{2(\hat{\sigma}^2)^2}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}) \\
\frac{n}{2\hat{\sigma}^2} &= \frac{1}{2(\hat{\sigma}^2)^2}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1}(y - X\hat{\boldsymbol{\theta}}) \\
\frac{2(\hat{\sigma}^2)^2}{n} \cdot \frac{n}{2\hat{\sigma}^2} &= \frac{2(\hat{\sigma}^2)^2}{n} \cdot \frac{1}{2(\hat{\sigma}^2)^2}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}) \\
\hat{\sigma}^2 &= \frac{1}{n}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top \mathbf{I}^{-1}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}}) \\
\hat{\sigma}^2 &= \frac{1}{n}(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\theta}})
\end{aligned}
$$

Масштабирование признаков

Рассмотрим линейную регрессию с двумя независимыми переменными с разным масштабом. Например, пусть один признак находится в диапазоне от 0 до 1000, а второй — от 0 до 1. Для того чтобы настроиться на такие признаки, модель выберет небольшой первый и большой второй коэффициенты.

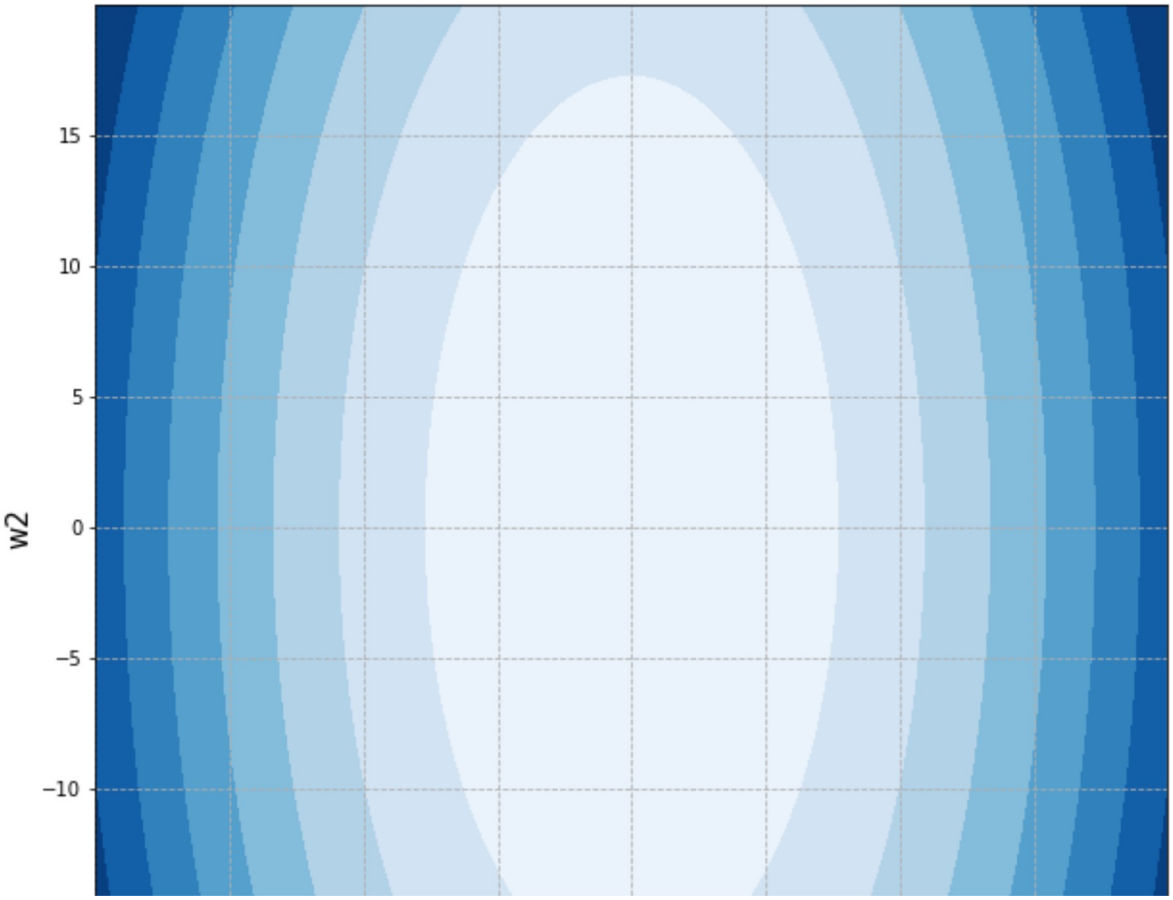
Проблема подобной модели в том, что на этапе обучения скорость оптимизации таких коэффициентов (весов, параметров) не будет одинаковой.

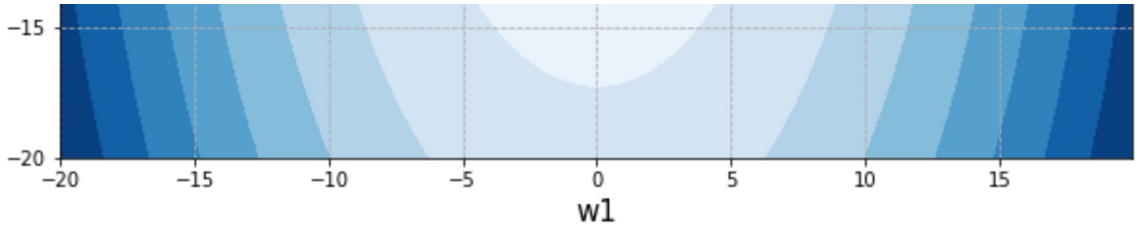
На графике изолиний (как бы «вид сверху») функция потерь такой модели примет эллиптическую форму (по оси x соответственно будет отложен признак с небольшим диапазоном, по оси y — с большим).

```
def paraboloid_countours(min, max, a = 1, b = 1, w1_list = Non

  # установим размер графика
  fig, ax = plt.subplots(figsize = (10,10))

  # создадим последовательность из 100 точек в интервале от -1
  # для осей w1 и w2
  w1_plot = np.arange(min, max, 0.02)
```

```
    w2_plot = np.arange(min, max, 0.02)

    # создадим координатную плоскость из осей w1 и w2
    w1_plot, w2_plot = np.meshgrid(w1_plot, w2_plot)

    f = (w1_plot ** 2)/a + (w2_plot ** 2)/b

    # построим изолинии (линии уровня)
    ax.contourf(w1_plot, w2_plot, f, cmap = 'Blues')

    # если функции передали списки весов, выведем траекторию их с
    if w1_list != None and w2_list != None:
      ax.plot(w1_list, w2_list, c = 'red')

    # укажем подписи к осям
    ax.set_xlabel('w1', fontsize = 15)
    ax.set_ylabel('w2', fontsize = 15)

    # создадим сетку в виде прерывистой черты
    plt.grid(linestyle = '--')

    # выведем результат
    plt.show()
```

```
paraboloid_countours(-20, 20, 1, 5)
```





При использовании алгоритма градиентного спуска мы в первую очередь найдем две частные производные этой функции и подберем *единый* для обеих производных коэффициент скорости обучения. Так как масштаб признаков будет различным, на каждой итерации мы будем получать существенно различающиеся значения градиента для разных направлений.

Пропишем функцию потерь и найдем ее частные производные.

```
# пропишем функцию потерь
# (переменные a и b будут определять масштаб каждого из признак
def objective(w1, w2, a, b):
    return (w1 ** 2) / a + (w2 ** 2) / b

# производную по первой
def partial_1(w1, a):
  return (2.0 * w1) / a

# и второй переменным
def partial_2(w2, b):
  return (2.0 * w2) / b
```

Попробуем найти оптимальные веса с помощью алгоритма градиентного спуска.

```
def paraboloid_gd(w1, w2, a, b, iter = 100, learning_rate = 0.

  # создадим списки для учета весов и уровня ошибки
  w1_list, w2_list, l_list = [], [], []

  # в цикле с заданным количеством итераций
  for i in range(iter):

    # будем добавлять текущие веса в соответствующие списки
    w1_list.append(w1)
    w2_list.append(w2)
```

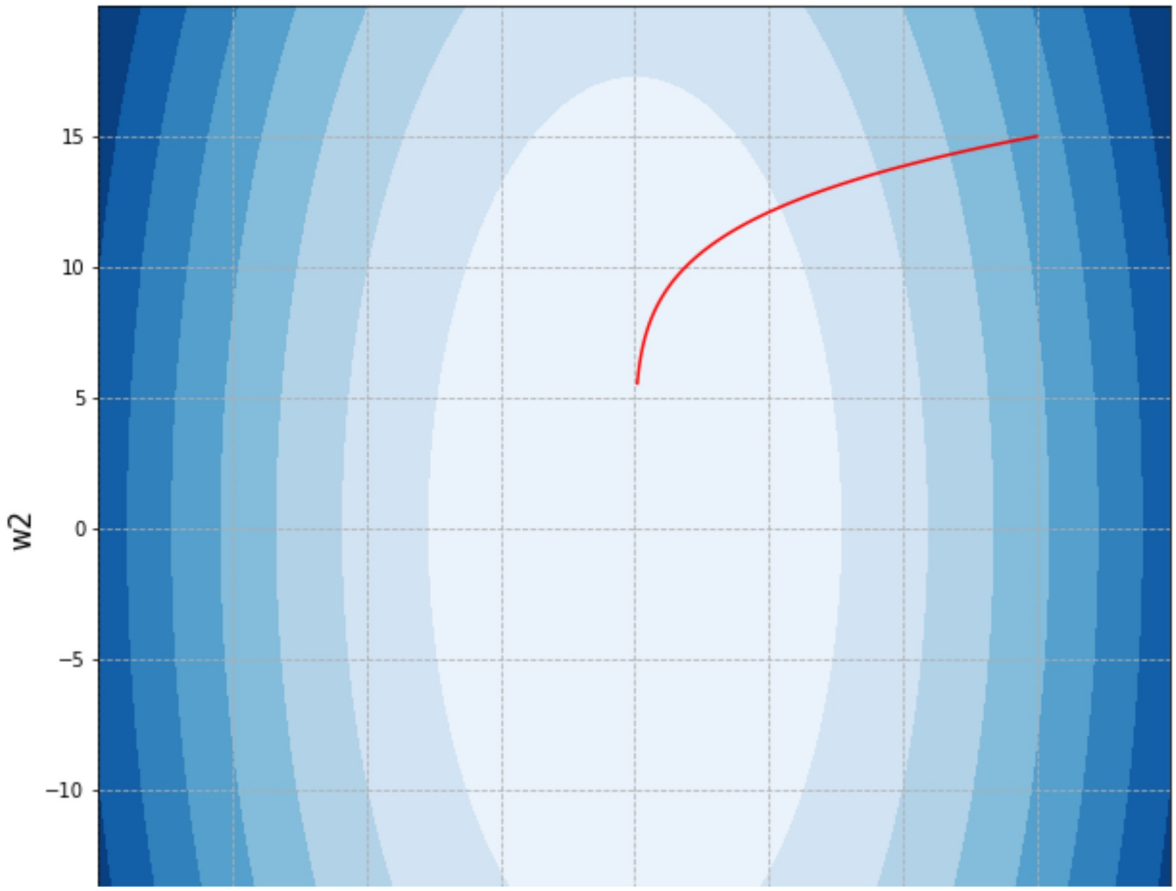
```
    # и рассчитывать и добавлять в список текущий уровень ошиб
    l_list.append(objective(w1, w2, a, b))

    # также рассчитаем значение частных производных при текущи
    par_1 = partial_1(w1, a)
    par_2 = partial_2(w2, b)

    # будем обновлять веса в направлении,
    # обратном направлению градиента, умноженному на скорость 
    w1 = w1 - learning_rate * par_1
    w2 = w2 - learning_rate * par_2

  # выведем итоговые веса модели и значение функции потерь
  return w1_list, w2_list, l_list
```

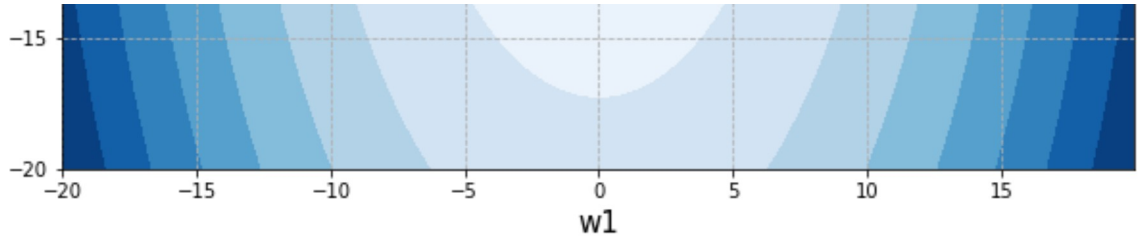
```
w1_list, w2_list, l_list = paraboloid_gd(15, 15, 1, 5)
w1_list[-1], w2_list[-1], l_list[-1]
```

```
(0.09348204032106357, 5.545944564745904, 6.1602391149095155)
```

Вновь выведем изолинии функции потерь, дополнив их траекторией изменения весов признаков.

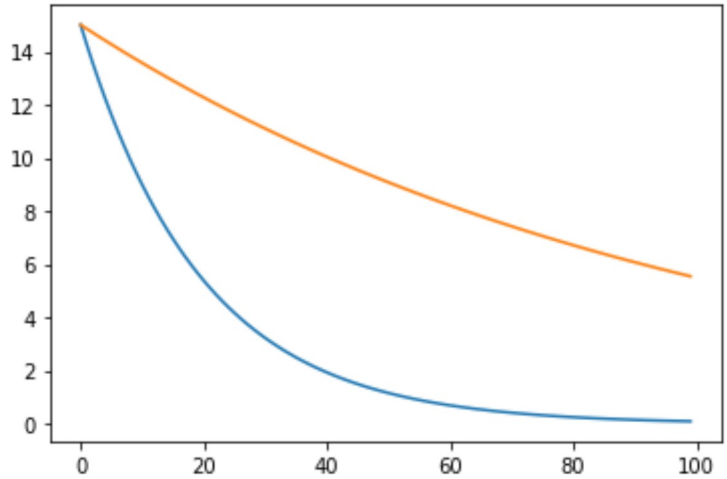
```
paraboloid_countours(-20, 20, 1, 5, w1_list, w2_list)
```





Мы не смогли дойти до минимума. Для иллюстрации этой особенности можно также вывести графики изменения весов в процессе обучения алгоритма.

```
plt.plot(w1_list)
plt.plot(w2_list)
plt.show()
```



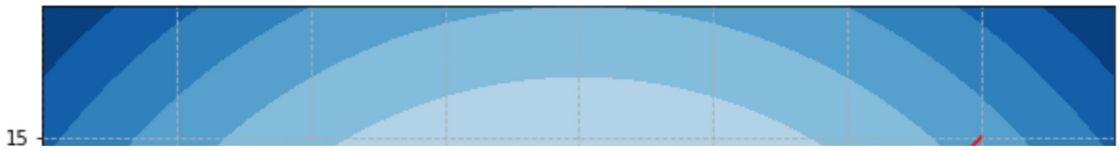
Посмотрим, что будет, если привести признаки к одному масштабу. Вначале найдем минимум методом градиентного спуска.

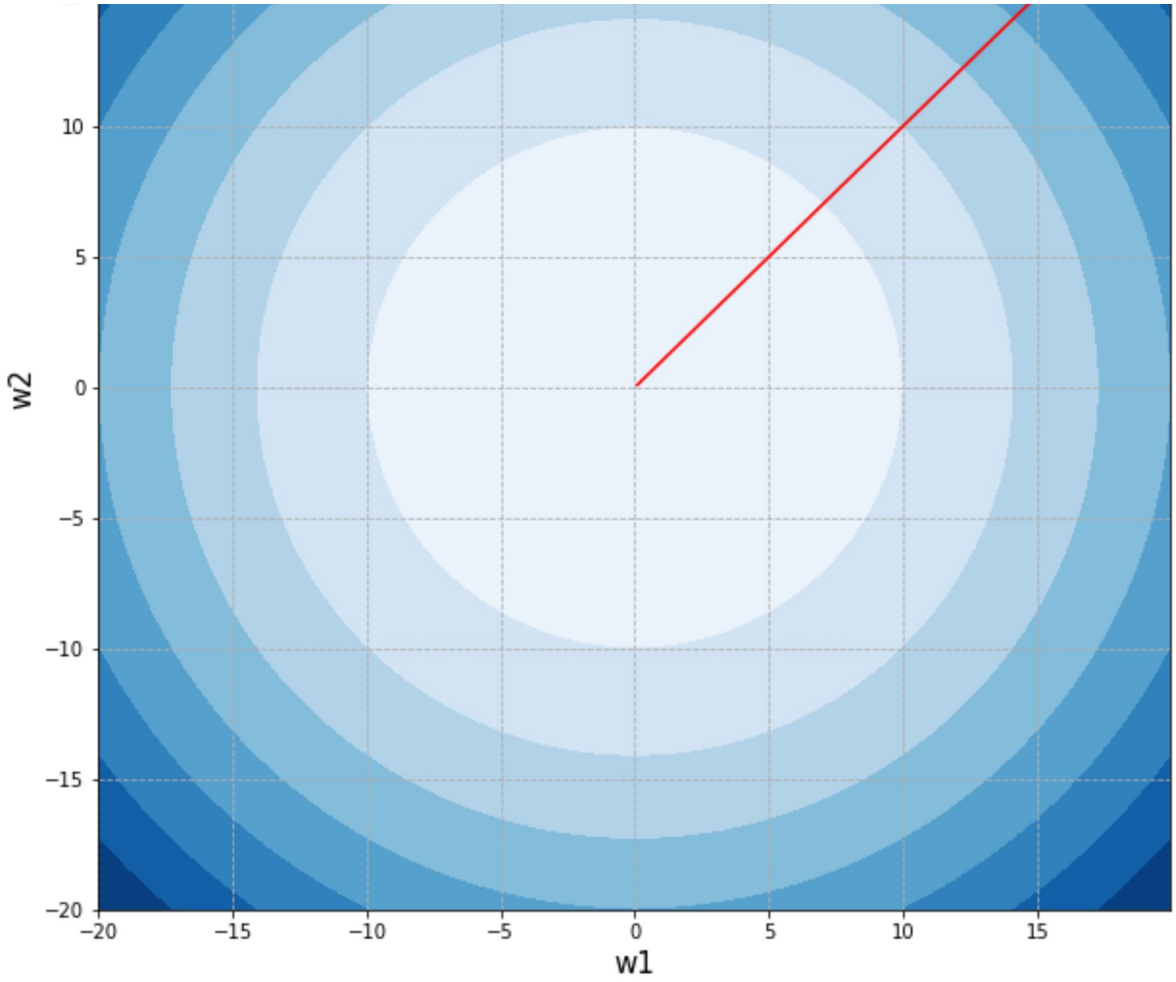
```
# напомню, параметры a и b в данном случае контролируют масштаб
w1_list, w2_list, l_list = paraboloid_gd(15, 15, 1, 1)
w1_list[-1], w2_list[-1], l_list[-1]
```

```
(0.09348204032106357, 0.09348204032106357, 0.017477783725177908
```

Выведем результат на графике.

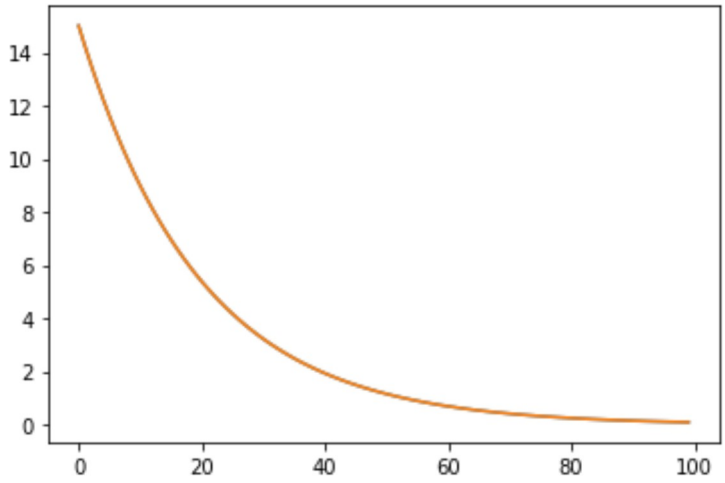
```
paraboloid_countours(-20, 20, 1, 1, w1_list, w2_list)
```





Посмотрим на обновления весов.

```
plt.plot(w1_list)
plt.plot(w2_list)
plt.show()
```



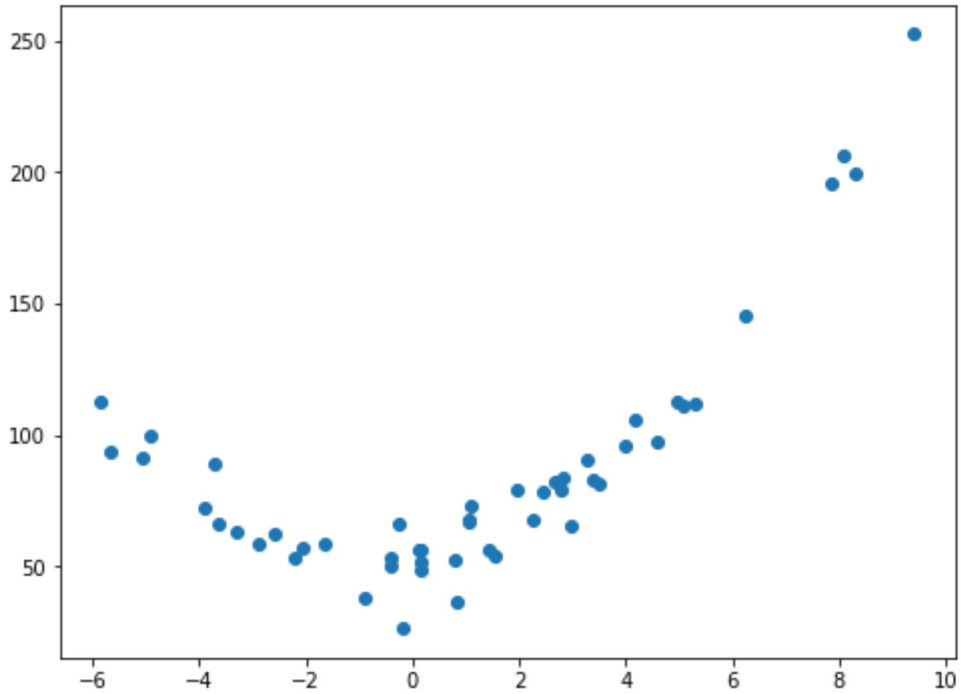
Мы видим только оранжевую линию, потому что одна кривая перекрывает другую.

Полиномиальная регрессия

Данные далеко не всегда хорошо аппроксимируются прямой линией. Создадим искусственный набор данных.

```
np.random.seed(42)
X = np.random.normal(2, 4, 50)
y = 2 * X ** 2 + 2 * X + 50 + np.random.normal(3, 10, 50)
```

```
plt.scatter(X, y);
```



Попробуем «наложить» на данные модель линейной регрессии.

```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X.reshape(-1, 1), y)
y_pred = model.predict(X.reshape(-1, 1))
```

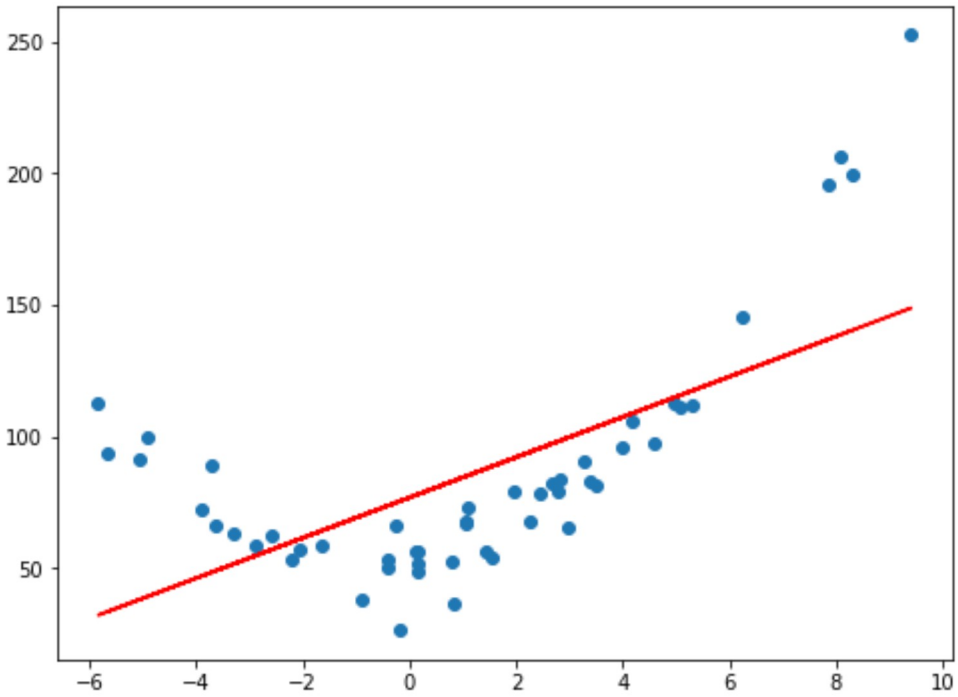
Посмотрим на получившиеся коэффициенты.

```
model.coef_, model.intercept_
```

```
(array([7.65521764]), 76.7171405919949)
```

Выведем результат на графике и оценим качество через RMSE и R^2.

```
plt.scatter(X, y)
plt.plot(X, y_pred, color='r')
plt.show()
```




```
from sklearn.metrics import root_mean_squared_error, r2_score

root_mean_squared_error(y, y_pred), r2_score(y, y_pred)
```

```
(34.5483663960937, 0.40159428156318455)
```

В данном случае очевидно, что данные гораздо более удачно моделировать с помощью квадратичной функции (quadratic function).

$$\hat{y} = \theta_0 + \theta_1 x + \theta_2 x^2$$

Обратите внимание, что в квадрат возводится не коэффициент θ, а признак x. Поэтому все, что нам нужно, это создать еще один признак, равный квадрату исходного признака. Для этого в sklearn есть **класс PolynomialFeatures**.

```
from sklearn.preprocessing import PolynomialFeatures

polynomial_features = PolynomialFeatures(degree = 2, include_bi
X_poly = polynomial_features.fit_transform(X.reshape(-1, 1))
```

Параметр include_bias регулирует, добавлять ли столбец из единиц

для свободного коэффициента или нет. Так как в данном случае мы используем класс LinearRegression, который сам рассчитывает все коэффициенты, такой столбец нам не понадобится.

```
# сравним исходный признак
X[:5]
```

```
array([3.98685661, 1.4469428 , 4.59075415, 8.09211943, 1.063386
```

```
# с новыми
X_poly[:5]
```

```
array([[ 3.98685661, 15.89502565],
       [ 1.4469428 ,  2.09364345],
       [ 4.59075415, 21.07502369],
       [ 8.09211943, 65.4823968 ],
       [ 1.0633865 ,  1.13079085]])
```

Второй столбец — квадрат первого.

```
3.98685661 ** 2
```

```
15.89502562870069
```

```
X.shape, X_poly.shape
```

```
((50,), (50, 2))
```

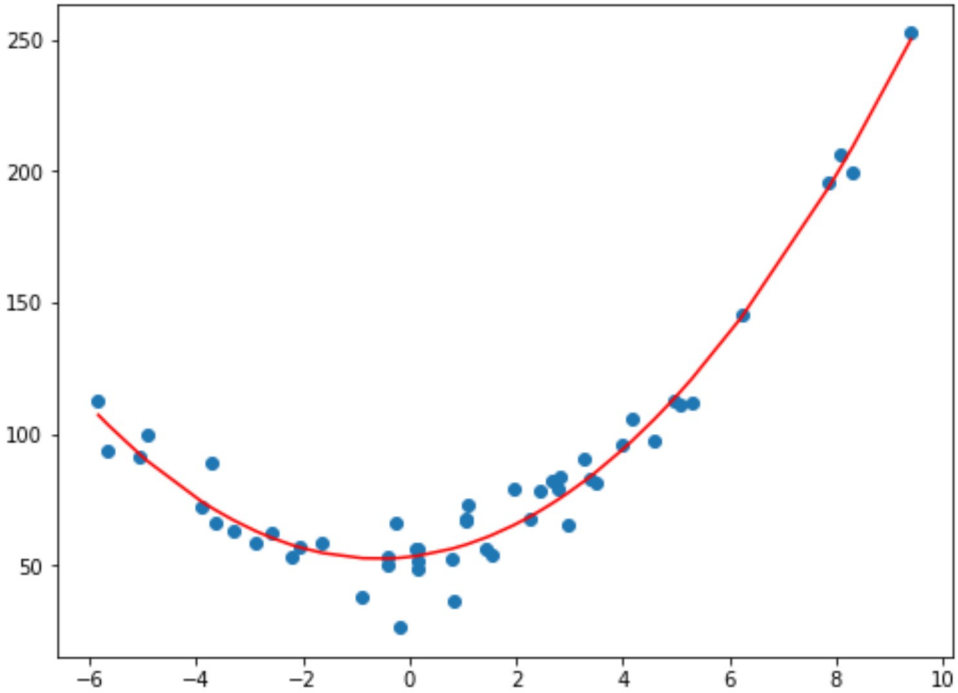
Построим модель.

```
model = LinearRegression()

model.fit(X_poly, y)
y_poly_pred = model.predict(X_poly)

model.coef_, model.intercept_
```

```
(array([2.31100218, 1.98024538]), 53.13013943148053)
```

```
import operator

sort_axis = operator.itemgetter(0)
sorted_zip = sorted(zip(X, y_poly_pred), key = sort_axis)
X_plot, y_poly_pred_plot = zip(*sorted_zip)

plt.plot(X_plot, y_poly_pred_plot, color = 'r')
plt.scatter(X, y)
```

```
9
10 plt.show()
```




```
1 root_mean_squared_error(y, y_poly_pred), r2_score(y, y_poly_pre
```

```
1 (8.596303829633031, 0.9629520471499875)
```

Два важных замечания:

- масштабирование признаков становится особенно значимым при оптимизации полиномиальных коэффициентов методом градиентного спуска (потому что, например, если мы возводим признак в квадрат, его диапазон также увеличивается аналогичным образом);
- функция не обязательно должна быть степенной; если нам нужно предсказать данные, которые растут, но затем выходят на плато (например, цены), можно использовать, в частности, фунцию квадратного корня.